

Module: Companion Pet

[pawguard/modules/companion_pet](#) — API prefix: `/companion-pets`

PawGuard Backend | FastAPI 3.13 + PostgreSQL 17 + Redis/ARQ + S3 | Generated from repo @ commit 99a8430 (2026-08-12)

1. Purpose

Owner-managed pet profiles (distinct from the shelter's Dog module): pet records, vet clinic directory, appointments, privacy-safe QR safety tags, medical file uploads, and vaccination/medication reminders. See the dedicated Pet Profile document set for full detail.

2. Position in the Layered Architecture

Client → **Router (auth+validate)** → **Service (business logic)** → **Repository (data access)** → **PostgreSQL**

Like every module in this codebase, business rules live only in the Service layer; Routers authenticate/authorize/validate and Repositories perform data access with no business decisions (see AGENTS.md — the project's engineering constitution).

3. Data Model — Owned Tables

Model Class	Table Name
CompanionPet	companion_pets
VetClinic	vet_clinics
ClinicMembership	vet_clinic_memberships
PetClinicAccess	pet_clinic_access
PetMedicalRecord	pet_medical_records
SafetyTag	pet_safety_tags
PetReminder	pet_reminders
ReminderDelivery	pet_reminder_deliveries
PetAppointment	pet_appointments

4. API Endpoints (28 total)

Method	Path	Permission Required
POST	<code>/companion-pets</code>	<code>companion_pet:create</code>
GET	<code>/companion-pets</code>	<code>companion_pet:read</code>
GET	<code>/companion-pets/clinics</code>	—
GET	<code>/companion-pets/clinics/{clinic_id}/veterinarians</code>	<code>appointment:read</code>
GET	<code>/companion-pets/appointments</code>	<code>appointment:read</code>
GET	<code>/companion-pets/{pet_id}</code>	<code>companion_pet:read</code>
PATCH	<code>/companion-pets/{pet_id}</code>	<code>companion_pet:update</code>
DELETE	<code>/companion-pets/{pet_id}</code>	<code>companion_pet:delete</code>
POST	<code>/companion-pets/{pet_id}/medical-files/upload-url</code>	<code>companion_pet:medical_upload</code>
PUT	<code>/companion-pets/{pet_id}/medical-files/{file_id}/confirm</code>	<code>companion_pet:medical_upload</code>
GET	<code>/companion-pets/{pet_id}/medical-files</code>	<code>companion_pet:read</code>
POST	<code>/companion-pets/{pet_id}/medical-records</code>	<code>companion_pet:medical_upload</code>
GET	<code>/companion-pets/{pet_id}/medical-records</code>	<code>companion_pet:read</code>

Method	Path	Permission Required
DELETE	/companion-pets/medical-records/{record_id}	companion_pet:medical_upload
POST	/companion-pets/{pet_id}/safety-tag	safety_tag:manage
GET	/companion-pets/{pet_id}/safety-tag	companion_pet:read
POST	/companion-pets/safety-tag/scan	—
POST	/companion-pets/clinics	vet_clinic:manage
PATCH	/companion-pets/clinics/{clinic_id}	vet_clinic:manage
DELETE	/companion-pets/clinics/{clinic_id}	vet_clinic:manage

+8 additional endpoints not shown — see *openapi.json* for the full list.

Distinct permission codes used: companion_pet:create, companion_pet:read, appointment:read, companion_pet:update, companion_pet:delete, companion_pet:medical_upload, safety_tag:manage, vet_clinic:manage, appointment:create, appointment:cancel, appointment:manage

5. Key Service-Layer Operations

- add_membership
- cancel_appointment
- confirm_medical_upload
- create_appointment
- create_clinic
- create_medical_record
- create_pet
- create_reminder
- delete_clinic
- delete_medical_record
- delete_pet
- delete_reminder
- deliver_reminder_once
- get_appointment

+16 additional service methods.

6. Notable Implementation Details

- QR safety tags store only a SHA-256 hash of the token; the raw token is returned once and never persisted.
- Two-layer clinic access: platform role + an explicit pet_clinic_access grant per pet/clinic pair.